MathScope v0.3.1

Master Implementation Checklist - A/B/C/D Detailed Build Plan

Implementation Blueprint

2026-10-07

목차

Implementation Dashboard **4**
0. 프로젝트 전체 Definition of Done . . . 5

1. Phase 0 - Core Migration & Foundation **6**
1.1 ResearchSession . . . 6

2. 공통 Typed Mathematical Object System **6**
2.1 ObjectSpec . . . 6
2.2 RepresentationSpec . . . 6

3. Revision / Dependency / Stale Engine **7**
3.1 Revision model . . . 7
3.2 stale propagation . . . 7
3.3 dependency test . . . 8

4. Evidence / Trust Ladder **8**
4.1 Evidence states . . . 8
4.2 EvidenceRecord . . . 8
4.3 금지 상태 transition . . . 8

5. IDE Shell **8**
5.1 Top bar . . . 9
5.2 Object Tree . . . 9
5.3 Main Canvas tabs . . . 9
5.4 Inspector . . . 9
5.5 Bottom Dock . . . 9

6. Explore Mode / Research Mode **10**
Explore . . . 10
Research . . . 10

7. v0.3.1-A - Analytic Core **11**
A1. Function Space Engine . . . 11

8. A2. Differential Systems Engine **11**
ODE . . . 11
DAE . . . 11
Linear PDE . . . 11
Nonlinear PDE . . . 11
PDE on manifolds . . . 12
Boundary conditions . . . 12
Initial conditions . . . 12

9. A3. PDESpec **12**

10. A4. PDE Numerical Solver Adapter **12**
입력 기록 . . . 12

출력 기록 . . . 13

11. A5. PDE Optimization Lab **13**
control candidates . . . 13
optimization . . . 13
UI . . . 13

12. A6. Operator Theory Lab **13**
필드 . . . 13
guard . . . 14

13. A7. Spectral Lab **14**
SpectrumSpec . . . 14
Point spectrum . . . 14
Continuous spectrum . . . 14
Residual spectrum . . . 14

14. A8. Resolvent Explorer **14**

15. A9. Finite Approximation Engine **15**

16. A10. Compact Self-Adjoint Spectral Engine **15**

17. A11. General Spectral Theorem Layer **15**

18. A12. Nonlinear PDE Bridge **16**
pipeline . . . 16
guard . . . 16

19. A Acceptance Gate **16**

20. v0.3.1-B - Geometry Engine **17**

21. B1. Singularity Lab **17**
상태 . . . 17
indicators . . . 17

22. B2. nD → 3D Engine **17**

23. B3. Fidelity Vector **17**

24. B4. Low-D → nD Lifting **18**
candidates . . . 18
각 candidate . . . 18

25. B5. Geometry Evolution Lab **18**
Ricci flow . . . 18
Surgery . . . 18

26. B6. Entropy Lab **18**
EntropySpec . . . 19

27. B7. Algebraic Topology / Persistence **19**

28. B Acceptance Gate **19**

29. v0.3.1-C - Research Bridge **20**

30. C1. Research Graph **20**
Node types . . . 20
Edge types . . . 20
UI . . . 20

31. C2. Research Copilot **20**
Copilot permissions . . . 20

32. C3. Stress Test / Counterexample Mode **21**
output . . . 21

33. C4. Group Theory Lab **21**

34. C5. Representation Theory **21**

35. C6. Galois Lab **22**

36. C7. Zeta-Resolvent Adapter **22**
allowed status . . . 22
forbidden . . . 22

37. C8. Proof Dock / Lean **22**
panels . . . 22
Diagnostics . . . 22
Final verifier . . . 23
상태 . . . 23

38. C9. Command Palette **23**

39. C Acceptance Gate **23**

40. v0.3.1-D - K-Theory / Index Layer **24**

41. D1. Differential / Pseudodifferential Operator **24**

42. D2. Principal Symbol Inspector **24**

43. D3. Ellipticity Checker **24**

44. D4. K-Theory Symbol Class **24**

45. D5. Index Lab **25**

46. D6. Atiyah-Singer Edge **25**

47. D7. Boundary / Atiyah-Bott Lab **25**
상태 . . . 26

48. D8. Nonlinear PDE → Index Bridge **26**

49. D9. Spectral Flow **26**

50. D10. Perturbation Lab **26**

51. D11. K-Homology **26**

52. D12. Equivariant K-Theory **27**

53. D Acceptance Gate **27**

54. Adapter Architecture **28**
모든 adapter 공통 . . . 28

55. Worker / Job System **28**

56. Computation Backend Integration **28**
ODE/DAE/nonlinear . . . 28
PDE/FEM . . . 28
Spectrum . . . 28
Topology . . . 29
3D . . . 29
Formal . . . 29

57. Local-first Architecture **29**

58. `.mathscope` Bundle **29**
Export . . . 29
Import . . . 30

59. Reproducibility **30**

60. Error & Uncertainty Panel **30**

61. Regression Test Suite **30**

62. Formalization Coverage Matrix **31**

63. Representative Demo Suite **32**
Demo 1 - S^3 Representation . . . 32
Demo 2 - Nonlinear PDE . . . 32
Demo 3 - Symmetric Operator . . . 32
Demo 4 - Elliptic Index . . . 32

64. 기준 Integration Test **33**
A . . . 33
B . . . 33
C . . . 33
D . . . 33

65. UI Routes **33**

66. Homepage **33**

67. Performance Checklist **34**

68. Failure UX **34**

69. Documentation Checklist **34**

70. 최종 Release Gate **35**
Product . . . 35
Mathematical . . . 35
Trust . . . 35
Formal . . . 35
Reproducibility . . . 35

권장 실제 개발 순서 **36**

Implementation Dashboard

Milestone	핵심 범위	상태
Foundation	ResearchSession / Schema / Revision / Evidence / IDE Shell	☐
v0.3.1-A	ODE/PDE / Operator / Spectrum / Resolvent	☐
v0.3.1-B	Singularity / nD->3D / Fidelity / Entropy / Topology	☐
v0.3.1-C	Research Graph / Stress Test / Symmetry / Lean	☐
v0.3.1-D	Symbol / Ellipticity / K-Class / Index / Boundary	☐
Integration & Release	Bundle / Benchmarks / Replay / Docs / Performance	☐

Tracking rule. 기능이 존재하는 것만으로 완료 처리하지 않는다. 해당 섹션의 guard, regression test, acceptance gate까지 PASS해야 체크한다.

Golden vertical slice. `Nonlinear Elliptic PDE` 세션 하나를 A -> B -> C -> D 전체로 먼저 관통시킨 뒤, Heat / Wave / Schrodinger / Ricci / Galois / zeta 등의 adapter를 확장한다.

0. 프로젝트 전체 Definition of Done

0.1 절대 규칙

- ☐ `Representation ≠ Object`를 데이터 구조 수준에서 강제
- ☐ `Visualized ≠ Proved`
- ☐ `Numerically Solved ≠ Exactly Solved`
- ☐ `Finite Truncation ≠ Infinite Operator`
- ☐ `High Fidelity ≠ Equivalence`
- ☐ `Linearized Stability ≠ Nonlinear Stability`
- ☐ `Spectral Correlation ≠ Spectral Identity`
- ☐ `3D Similarity ≠ Topological Equivalence`
- ☐ `Index = 0 ≠ Invertible`
- ☐ `AI Suggestion ≠ Theorem`

이 원칙들은 단순 경고문이 아니라 **type system + status transition + UI badge + regression test** 네 층에서 모두 구현한다. 설계도에서도 이 구분을 v0.3.1의 수학적 헌법으로 정의한다.

---

1. Phase 0 - Core Migration & Foundation

설계도에서 가장 먼저 요구하는 P0 영역이다.

1.1 ResearchSession

- ☐ `ResearchSession` 타입 생성
- ☐ `id`
- ☐ `title`
- ☐ `goals[]`
- ☐ `objects[]`
- ☐ `activeObjectRef`
- ☐ `representations[]`
- ☐ `operators[]`
- ☐ `pdes[]`
- ☐ `spectra[]`
- ☐ `geometries[]`
- ☐ `topologies[]`
- ☐ `kclasses[]`
- ☐ `claims[]`
- ☐ `evidence[]`
- ☐ `assumptions[]`
- ☐ `revisions[]`
- ☐ `checkpoints[]`
- ☐ `environmentRef`

이 공통 데이터 모델은 설계도의 핵심이다.

세션 동작

- ☐ Create Session
- ☐ Save Session
- ☐ Load Session
- ☐ Duplicate Session
- ☐ Rename Session
- ☐ Delete/Archive Session
- ☐ Session revision 생성
- ☐ checkpoint 생성
- ☐ checkpoint restore
- ☐ autosave
- ☐ dirty-state indicator
- ☐ crash recovery
- ☐ import/export

---

2. 공통 Typed Mathematical Object System

2.1 ObjectSpec

- ☐ type
- ☐ base field
- ☐ intrinsic dimension
- ☐ ambient dimension
- ☐ structure metadata
- ☐ domain/manifold reference
- ☐ revision id
- ☐ assumptions
- ☐ provenance
- ☐ hash

2.2 RepresentationSpec

- ☐ sourceObjectRef
- ☐ method
- ☐ displayDim

- ☐ mapDefinition
- ☐ parameters
- ☐ sampling
- ☐ injectivityStatus
- ☐ ambiguityRef
- ☐ fidelityVector
- ☐ provenance
- ☐ revision

설계도 Appendix B의 구조를 그대로 schema로 만드는 것이 좋다.

Representation method enum

- ☐ projection
- ☐ slice
- ☐ embedding
- ☐ stereographic
- ☐ PCA-like reduction
- ☐ spectral reduction
- ☐ user-defined map
- ☐ sampled finite representation

필수 guard

- ☐ 3D view를 생성할 때 RepresentationSpec 없이는 render 금지
- ☐ known non-injective projection이면 badge 자동 표시
- ☐ lost dimension 기록
- ☐ discarded coordinates 기록
- ☐ inverse 존재 여부
- ☐ ambiguity fibre 지원

---

3. Revision / Dependency / Stale Engine

이 부분이 MathScope의 차별점 중 하나다.

설계도에서는 object revision이 변경될 때 formal proof까지 무조건 삭제하지 않고, 이전 revision에 대한 `HISTORICAL FORMAL`로 보존하도록 규정한다.

3.1 Revision model

- ☐ 모든 핵심 node에 revision id
- ☐ parent revision
- ☐ diff/delta
- ☐ timestamp
- ☐ author/system source
- ☐ environment hash
- ☐ invalidated edges
- ☐ superseded edges

3.2 stale propagation

예:

```
Object X r1
  ↓
Representation R1
  ↓
Claim C1
  ↓
Evidence E1

Object X → r2
```

- ☐ R1 → `STALE`
- ☐ C1 → `REVIEW`
- ☐ numerical E1 → `STALE`

- ☐ formal proof → `HISTORICAL_FORMAL`
- ☐ previous theorem edge 보존
- ☐ 현재 revision을 지원하지 않는다는 경고

3.3 dependency test

- ☐ object change
- ☐ assumption change
- ☐ representation change
- ☐ mesh change
- ☐ solver tolerance change
- ☐ boundary condition change
- ☐ operator domain change
- ☐ Lean environment change

각 경우 어떤 downstream evidence가 stale 되는지 테스트 작성.

---

4. Evidence / Trust Ladder

설계도는 상태를 단순 성공/실패로 처리하지 않고 여러 증거 등급으로 분리한다.

4.1 Evidence states

- ☐ `FORMAL_PASS`
- ☐ `THEOREM-BACKED`
- ☐ `CERTIFIED_NUMERICAL`
- ☐ `NUMERICAL_INDICATOR`
- ☐ `EMPIRICAL_CORRESPONDENCE`
- ☐ `RESEARCH_HYPOTHESIS`
- ☐ `UNKNOWN`
- ☐ `STALE`

설계도 정의와 일치시킨다.

4.2 EvidenceRecord

- ☐ id
- ☐ grade
- ☐ claimRef
- ☐ method
- ☐ inputsHash
- ☐ environmentHash
- ☐ residuals
- ☐ errorBounds
- ☐ assumptions
- ☐ generatedAt
- ☐ adapterVersion
- ☐ upstreamRevisions[]
- ☐ stale flag

원본 schema가 설계도 Appendix B에 있다.

4.3 금지 상태 transition

- ☐ AI → FORMAL PASS 직접 금지
- ☐ numerical → THEOREM-BACKED 자동 승격 금지
- ☐ visual → topology equivalence 금지
- ☐ LSP no-error → FORMAL PASS 금지
- ☐ large-N spectrum → exact infinite spectrum 금지

---

5. IDE Shell

5.1 Top bar

- ☐ project/session selector
- ☐ current goal
- ☐ Run
- ☐ Verify
- ☐ Stress Test
- ☐ Command Palette
- ☐ evidence summary
- ☐ environment status

5.2 Object Tree

- ☐ mathematical object
- ☐ representations
- ☐ function spaces
- ☐ operators
- ☐ PDE
- ☐ spectra
- ☐ geometry
- ☐ topology
- ☐ entropy
- ☐ symmetry
- ☐ K-theory
- ☐ claims
- ☐ evidence

5.3 Main Canvas tabs

- ☐ Field
- ☐ 2D
- ☐ 3D
- ☐ Spectrum
- ☐ Resolvent
- ☐ Singularity
- ☐ Geometry
- ☐ Topology
- ☐ Group
- ☐ Index
- ☐ Research Graph

5.4 Inspector

- ☐ type
- ☐ current revision
- ☐ assumptions
- ☐ representation method
- ☐ fidelity
- ☐ evidence grade
- ☐ provenance
- ☐ stale state
- ☐ numerical errors
- ☐ theorem assumptions

5.5 Bottom Dock

설계도는 PDE / Operator / Spectrum / K-Theory / Lean / Claims / Evidence / Console / Dependency Graph 구조를 제안한다.

- ☐ PDE
- ☐ Operator
- ☐ Spectrum
- ☐ K-Theory
- ☐ Lean
- ☐ Claims

☐ Evidence
☐ Console
☐ Dependency Graph

---

6. Explore Mode / Research Mode

Explore

☐ Object
☐ 3D view
☐ what is preserved?
☐ what is lost?
☐ simple status
☐ simplified controls

Research

☐ ObjectSpec
☐ RepresentationSpec
☐ assumptions Γ
☐ revisions
☐ hashes
☐ fidelity vector
☐ evidence
☐ dependency graph
☐ formal environment

두 모드는 수학적 backend가 달라지는 게 아니라 **노출 복잡도만 달라져야 한다.**

---

7. v0.3.1-A - Analytic Core

이제 핵심 계산 계층이다.

설계도에서 A는 함수공간·연산자 domain·PDE·discretization·spectrum·resolvent를 typed object로 만드는 단계다.

A1. Function Space Engine

- ☐ L^p
- ☐ L^2
- ☐ Sobolev H^s
- ☐ H^1
- ☐ H^2
- ☐ Banach
- ☐ Hilbert
- ☐ finite-dimensional vector spaces
- ☐ custom normed space

FunctionSpaceSpec

- ☐ scalar field
- ☐ domain
- ☐ norm
- ☐ inner product
- ☐ measure
- ☐ completeness status
- ☐ boundary regularity assumptions
- ☐ basis/frame reference

---

8. A2. Differential Systems Engine

설계도 지원범위를 그대로 구현한다.

ODE

- ☐ linear
- ☐ nonlinear
- ☐ autonomous
- ☐ non-autonomous
- ☐ systems
- ☐ parameterized ODE
- ☐ Jacobian
- ☐ stiffness metadata

DAE

- ☐ differential variables
- ☐ algebraic constraints
- ☐ index metadata
- ☐ consistency checks

Linear PDE

- ☐ elliptic
- ☐ parabolic
- ☐ hyperbolic
- ☐ mixed/unknown classification

Nonlinear PDE

- ☐ semilinear
- ☐ quasilinear
- ☐ selected fully nonlinear

☐ nonlinear term representation
☐ linearization hook

PDE on manifolds

☐ manifold reference
☐ metric
☐ charts
☐ mesh
☐ Laplace-Beltrami
☐ connection-dependent operators

Boundary conditions

☐ Dirichlet
☐ Neumann
☐ Robin
☐ periodic
☐ custom
☐ spectral/nonlocal placeholder

Initial conditions

☐ symbolic
☐ sampled
☐ mesh field
☐ file imported

---

9. A3. PDESpec

☐ equation
☐ classification
☐ manifold/domain reference
☐ function space
☐ coefficients
☐ parameters
☐ boundaryConditions[]
☐ initialData
☐ nonlinear flag
☐ linearizationRule
☐ objectiveRef
☐ solverPlan
☐ evidencePolicy

설계도 schema와 맞춘다.

---

10. A4. PDE Numerical Solver Adapter

입력 기록

☐ PDE revision
☐ mesh revision
☐ BC revision
☐ IC revision
☐ timestep
☐ solver
☐ tolerance
☐ preconditioner
☐ max iterations

출력 기록

- ☐ solution field
- ☐ residual norm
- ☐ convergence
- ☐ iterations
- ☐ elapsed time
- ☐ warning
- ☐ solver log
- ☐ environment hash

---

11. A5. PDE Optimization Lab

설계도 표준형:

$$\min_{m,u} J(u,m) \quad \text{s.t.} \quad F(u,m)=0$$

이다.

control candidates

- ☐ coefficient
- ☐ source
- ☐ initial data
- ☐ boundary parameter
- ☐ domain geometry
- ☐ metric
- ☐ physical parameter

optimization

- ☐ objective function
- ☐ regularizer
- ☐ bounds
- ☐ constraints
- ☐ gradient method
- ☐ finite difference
- ☐ automatic differentiation
- ☐ adjoint
- ☐ convergence criterion

UI

- ☐ manual slider exploration
- ☐ `Optimize` button
- ☐ distinguish explore/run
- ☐ objective history plot
- ☐ residual history
- ☐ parameter trajectory
- ☐ solution comparison

---

12. A6. Operator Theory Lab

설계도 OperatorSpec를 그대로 구현.

필드

- ☐ operator T
- ☐ domain $D(T)$
- ☐ codomain
- ☐ base field

- ☐ linear
- ☐ bounded
- ☐ densely defined
- ☐ closed
- ☐ closable
- ☐ symmetric
- ☐ self-adjoint
- ☐ normal
- ☐ unitary
- ☐ compact
- ☐ adjointRef
- ☐ norm
- ☐ spectrumRef
- ☐ resolventRef

guard

- ☐ unbounded operator인데 domain 없음 → INVALID
- ☐ symmetric → self-adjoint 자동 promotion 금지
- ☐ adjoint domain 별도 저장
- ☐ closure 정보 저장
- ☐ finite matrix representation을 operator identity로 오인 금지

---

13. A7. Spectral Lab

설계도는 point/continuous/residual/resolvent/pseudospectrum을 분리한다.

SpectrumSpec

- ☐ point spectrum
- ☐ continuous spectrum
- ☐ residual spectrum
- ☐ resolvent
- ☐ pseudospectrum
- ☐ spectral radius
- ☐ mode: exact/theorem/finiteApprox
- ☐ truncation N
- ☐ convergence series
- ☐ pollution risk

Point spectrum

- ☐ eigenvalues
- ☐ eigenvectors
- ☐ multiplicity
- ☐ residual

Continuous spectrum

- ☐ classification criteria
- ☐ evidence
- ☐ region rendering

Residual spectrum

- ☐ range density status
- ☐ classification evidence

---

14. A8. Resolvent Explorer

$$R(\lambda, T) = (\lambda I - T)^{-1}$$

- ☐ complex λ cursor
- ☐ interactive complex plane
- ☐ $\|R(\lambda, T)\|$
- ☐ spectrum overlay
- ☐ singular points
- ☐ level sets
- ☐ pseudospectrum
- ☐ normal operator mode
- ☐ non-normal operator mode
- ☐ finite discretization warning

설계도는 이 기능을 별도로 정의한다.

---

15. A9. Finite Approximation Engine

$$P_N T P_N = A_N$$

- ☐ approximation node 생성
- ☐ parent operator reference
- ☐ N
- ☐ basis
- ☐ projection
- ☐ eigenvalue convergence
- ☐ eigenvector convergence
- ☐ residual
- ☐ spectral pollution risk
- ☐ projection error

UI

큰 배너:

FINITE APPROXIMATION OF AN INFINITE-DIMENSIONAL OPERATOR

- ☐ exact badge 금지
- ☐ N 항상 표시
- ☐ basis 표시
- ☐ convergence panel

---

16. A10. Compact Self-Adjoint Spectral Engine

- ☐ finite matrix diagonalization
- ☐ eigenbasis
- ☐ infinite eigenmode model
- ☐ 0 accumulation behavior
- ☐ modal coefficient visualization
- ☐ PDE solution modal decomposition
- ☐ truncation sequence

---

17. A11. General Spectral Theorem Layer

추상 representation:

$$T = \int_{\sigma(T)} \lambda \, dE(\lambda)$$

- ☐ spectral measure node
- ☐ projection-valued measure metadata
- ☐ distinguish abstract theorem vs computation
- ☐ functional calculus

$$f(T) = \int f(\lambda) dE(\lambda)$$

☐ heat e^{-tT}
☐ Schrödinger e^{-itT}
☐ custom $f(T)$

---

18. A12. Nonlinear PDE Bridge

설계도는 비선형 PDE에 spectral theorem을 곧바로 적용하는 것을 금지한다.

pipeline

☐ $F(u) = 0$
☐ candidate u_*
☐ $DF(u_*)$
☐ linearized OperatorSpec
☐ spectrum
☐ Galerkin
☐ bifurcation indicators
☐ nonlinear solver

guard

☐ linearized stability badge
☐ nonlinear stability badge와 분리
☐ global stability 자동 promotion 금지

---

19. A Acceptance Gate

설계도 A1-A6를 자동 테스트로 만든다.

☐ **A1** Operator typing
☐ **A2** PDE provenance
☐ **A3** Spectrum classification
☐ **A4** Infinite vs finite
☐ **A5** Nonlinear boundary
☐ **A6** Reproducibility

여섯 개 전부 PASS 전에는 v0.3.1-B default-on 금지.

---

20. v0.3.1-B - Geometry Engine

B의 목표는 A의 수학 객체를 고차원 공간과 3D representation 사이에서 이동시키되 정보 손실을 추적하는 것이다.

---

21. B1. Singularity Lab

상태

- ☐ REGULAR
- ☐ POSSIBLY SINGULAR
- ☐ NUMERICALLY SINGULAR
- ☐ THEOREM-BACKED SINGULAR
- ☐ FORMALLY PROVED
- ☐ UNKNOWN

설계도 상태 정의 그대로.

indicators

- ☐ $\|u\|_{L^\infty}$
- ☐ $\|\nabla u\|$
- ☐ $\|u\|_{H^s}$
- ☐ curvature
- ☐ Jacobian determinant
- ☐ residual
- ☐ energy concentration
- ☐ smallest singular value
- ☐ refinement persistence

---

22. B2. nD → 3D Engine

- ☐ source M^n
- ☐ projection π
- ☐ displayed object
- ☐ lost coordinates
- ☐ injectivity
- ☐ ambiguity
- ☐ source singular set S
- ☐ displayed $\pi(S)$

절대 금지

- ☐ $\pi(S)$만 보고 S 복원됐다고 표시 금지

---

23. B3. Fidelity Vector

설계도는 하나의 0-100 점수로 뭉개지 말라고 명시한다.

- ☐ Hausdorff defect
- ☐ topology defect
- ☐ persistence defect
- ☐ curvature defect
- ☐ spectral defect
- ☐ singularity-location defect
- ☐ energy defect

추가 구현

- ☐ each metric raw value
- ☐ normalization
- ☐ tolerance

- ☐ status
- ☐ assumptions
- ☐ source revision
- ☐ representation revision

---

24. B4. Low-D → nD Lifting

설계도는 이것을 역문제로 취급한다.

candidates

- ☐ product
- ☐ warped product
- ☐ fibration
- ☐ symmetry extension
- ☐ radial extension
- ☐ spectral extension

각 candidate

- ☐ assumptions
- ☐ non-uniqueness warning
- ☐ geometry
- ☐ spectrum
- ☐ topology
- ☐ fidelity
- ☐ evidence

---

25. B5. Geometry Evolution Lab

Ricci flow

$$\partial_t g = -2\,\mathrm{Ric}(g)$$

- ☐ metric animation
- ☐ scalar curvature
- ☐ Ricci tensor summary
- ☐ volume
- ☐ selected spectrum
- ☐ entropy
- ☐ singular region

Surgery

- ☐ surgery event
- ☐ pre-event geometry
- ☐ post-event geometry
- ☐ topology revision
- ☐ discontinuity marker

guard

- ☐ surgery를 dimension-lifting justification으로 자동 사용 금지

---

26. B6. Entropy Lab

설계도는 entropy 정의들을 섞지 않는다.

- ☐ Shannon
- ☐ metric
- ☐ topological
- ☐ volume

☐ spectral
☐ Perelman $\mathcal{F}$
☐ Perelman $\mathcal{W}$
☐ von Neumann

EntropySpec

☐ definition
☐ measure
☐ state/distribution
☐ normalization
☐ units if applicable
☐ assumptions

---

27. B7. Algebraic Topology / Persistence

☐ level set extraction
☐ connected components
☐ H_0
☐ H_1
☐ H_2
☐ Betti numbers
☐ persistent homology
☐ barcode
☐ persistence diagram
☐ pre/post projection comparison

---

28. B Acceptance Gate

설계도 B1-B6.

☐ B1 Projection typing
☐ B2 Singularity status
☐ B3 Fidelity vector
☐ B4 Lifting non-uniqueness
☐ B5 Entropy semantics
☐ B6 Topology check

---

29. v0.3.1-C - Research Bridge

C는 계산 기능을 **연구 workflow**로 만드는 계층이다.

---

30. C1. Research Graph

Node types

- ☐ Object
- ☐ Representation
- ☐ Function Space
- ☐ Operator
- ☐ PDE
- ☐ Spectrum
- ☐ Geometry
- ☐ Topology
- ☐ Symmetry
- ☐ K-class
- ☐ Claim
- ☐ Evidence
- ☐ Assumption
- ☐ Revision

Edge types

- ☐ representedBy
- ☐ actsOn
- ☐ generatedBy
- ☐ linearizesTo
- ☐ spectrumOf
- ☐ topologyOf
- ☐ derivedFrom
- ☐ supports
- ☐ dependsOn
- ☐ invalidates
- ☐ supersedes
- ☐ theorem-backed
- ☐ numerical-only

UI

- ☐ node inspect
- ☐ edge inspect
- ☐ hide/show evidence grades
- ☐ stale highlighting
- ☐ revision history
- ☐ upstream dependencies
- ☐ downstream dependencies

---

31. C2. Research Copilot

설계도에서 AI는 generator가 아니라 orchestrator다.

Copilot permissions

허용:

- ☐ suggest representation
- ☐ suggest solver
- ☐ suggest theorem
- ☐ suggest stress test
- ☐ suggest Lean target

☐ explain assumptions
☐ create draft claim

금지:

☐ set `FORMAL PASS`
☐ set `THEOREM-BACKED` without theorem map
☐ delete contradictory evidence
☐ suppress failed tests
☐ infer equivalence from visualization

---

32. C3. Stress Test / Counterexample Mode

설계도 항목을 모두 구현한다.

☐ identifiability failure search
☐ coordinate dependence
☐ basis dependence
☐ mesh refinement
☐ timestep refinement
☐ truncation N
☐ parameter perturbation
☐ assumption removal
☐ adversarial edge cases
☐ solver divergence test
☐ symmetry-breaking perturbation

output

☐ COUNTEREXAMPLE FOUND
☐ NUMERICAL WARNING
☐ INCONCLUSIVE
☐ PASSED SELECTED TESTS

절대로:

☐ “No counterexample found ⇒ theorem”

으로 승격하지 않는다.

---

33. C4. Group Theory Lab

☐ group definition
☐ generators
☐ relations
☐ Cayley graph
☐ orbit
☐ stabilizer
☐ fixed set
☐ subgroup lattice

---

34. C5. Representation Theory

☐ $\rho : G \to U(H)$
☐ representation validity
☐ invariant subspaces
☐ irreducible decomposition placeholder
☐ operator commutation

$$T\rho(g) = \rho(g)T$$

☐ numerical commutation check
☐ theorem-backed commutation
☐ spectral multiplicity comparison

---

35. C6. Galois Lab

설계도는 명시적인 field extension과 group structure를 요구한다.

☐ polynomial
☐ roots
☐ splitting field
☐ field extension
☐ Galois group
☐ subgroup lattice
☐ field tower
☐ Galois correspondence
☐ dual synchronized view

guard

☐ 3D resemblance → Galois relation 금지

---

36. C7. Zeta-Resolvent Adapter

설계도에서 Riemann zeta와 spectral zeta를 반드시 분리한다.

☐ Riemann $\zeta(s)$
☐ spectral $\zeta_T(s)$
☐ spectral counting
☐ resolvent
☐ eigenvalue statistics
☐ zero statistics comparison

allowed status

☐ EMPIRICAL CORRESPONDENCE
☐ RESEARCH HYPOTHESIS

forbidden

☐ matching statistics ⇒ spectral identity

---

37. C8. Proof Dock / Lean

설계도 정의가 아주 명확하다.

panels

☐ Natural Statement
☐ Formal Statement
☐ Diagnostics
☐ Final Verification
☐ Axiom Audit
☐ Environment
☐ Coverage

Diagnostics

☐ editor errors
☐ syntax
☐ type errors
☐ LSP status

Final verifier

☐ pinned Lean version
☐ pinned mathlib
☐ build command
☐ hash
☐ stdout/stderr
☐ final exit code
☐ proof result

상태

☐ FORMAL PASS
☐ FORMAL FAIL
☐ PARTIAL
☐ DEPENDENCY MISSING
☐ INFRA HOLD

---

38. C9. Command Palette

설계도 기능 전부.

☐ Create mathematical object
☐ Add representation
☐ Add operator
☐ Linearize operator
☐ Create PDE
☐ Add boundary condition
☐ Run spectral analysis
☐ Run stress test
☐ Create Lean target
☐ Inspect assumptions
☐ Open dependency graph
☐ Export session

---

39. C Acceptance Gate

☐ C1 Dependency Graph
☐ C2 AI Boundary
☐ C3 Stress Test
☐ C4 Symmetry Semantics
☐ C5 Zeta Semantics
☐ C6 Formal Boundary

설계도 요구사항 그대로다.

---

40. v0.3.1-D - K-Theory / Index Layer

이 계층의 목표는 PDE를 "풀기 전에 구조적으로 검사"하는 것이다.

---

41. D1. Differential / Pseudodifferential Operator

- ☐ operator order
- ☐ bundles E, F
- ☐ domain manifold
- ☐ differential order
- ☐ principal part
- ☐ symbol generation hook

---

42. D2. Principal Symbol Inspector

$$\sigma_m(D)(x,\xi)$$

- ☐ base point x
- ☐ cotangent ξ
- ☐ normalize $|\xi| = 1$
- ☐ matrix representation
- ☐ determinant
- ☐ smallest singular value
- ☐ eigenvalue phase
- ☐ invertibility
- ☐ characteristic locus

설계도 ellipticity 정의를 그대로 지킨다.

---

43. D3. Ellipticity Checker

- ☐ theorem mode
- ☐ symbolic mode
- ☐ sampled numerical mode
- ☐ local warning
- ☐ characteristic direction found
- ☐ incomplete sampling warning

status

- ☐ THEOREM-BACKED ELLIPTIC
- ☐ CERTIFIED NUMERICAL
- ☐ NUMERICALLY CONSISTENT
- ☐ FAIL
- ☐ UNKNOWN

---

44. D4. K-Theory Symbol Class

$$[\sigma(D)] \in K_c^0(T^*M)$$

- ☐ KTheorySpec node
- ☐ principalSymbolRef
- ☐ symbolClassRef
- ☐ theorem assumptions
- ☐ compact support semantics
- ☐ representative
- ☐ invariant data

visual

- ☐ sampled symbol field
- ☐ determinant phase
- ☐ eigenbundle visualization
- ☐ characteristic locus

permanent warning

> DISPLAY IS NOT THE K-CLASS ITSELF

이 구분은 설계도에도 명시되어 있다.

---

45. D5. Index Lab

$$\operatorname{ind}_a D = \dim \ker D - \dim \operatorname{coker} D$$

- ☐ kernel
- ☐ cokernel
- ☐ analytic index
- ☐ topological index
- ☐ theorem assumptions
- ☐ comparison
- ☐ mismatch diagnostics

설계도 규칙:

- ☐ index=0 ≠ invertible
- ☐ index alone ≠ solvability
- ☐ kernel/cokernel separately tracked

---

46. D6. Atiyah-Singer Edge

Research Graph:

```
Symbol K-Class
      ↓
Topological Index
      ↔
Analytic Index
```

- ☐ theorem assumptions
- ☐ compactness
- ☐ ellipticity
- ☐ manifold conditions
- ☐ operator class
- ☐ theorem source
- ☐ evidence status

---

47. D7. Boundary / Atiyah-Bott Lab

설계도가 특별히 경고하는 부분이다.

- ☐ manifold boundary
- ☐ boundary symbol
- ☐ boundary class
- ☐ local elliptic BC eligibility
- ☐ obstruction
- ☐ Dirichlet compatibility
- ☐ Neumann compatibility
- ☐ Robin compatibility
- ☐ APS option
- ☐ spectral/global option

상태

```
Atiyah-Bott obstruction: NONZERO
Local elliptic BC: OBSTRUCTED
APS/global alternative: POSSIBLE
```

절대로:

> ALL BOUNDARY CONDITIONS IMPOSSIBLE

라고 말하지 않는다.

---

48. D8. Nonlinear PDE → Index Bridge

설계도 pipeline 그대로.

$$F(u) = 0$$

- ☐ candidate u_*
- ☐ $L = DF(u_*)$
- ☐ ellipticity
- ☐ principal symbol
- ☐ K-class
- ☐ Fredholm index
- ☐ kernel dimension
- ☐ cokernel dimension
- ☐ local moduli dimension indicator
- ☐ bifurcation warning

---

49. D9. Spectral Flow

$$D_t$$

- ☐ operator family
- ☐ self-adjoint property
- ☐ Fredholm condition
- ☐ parameter t
- ☐ eigenvalue tracks
- ☐ zero crossings
- ☐ positive crossings
- ☐ negative crossings
- ☐ spectral flow

설계도 예시는 +4, -1, SF=3.

---

50. D10. Perturbation Lab

- ☐ D
- ☐ $D + K_1$
- ☐ $D + K_2$
- ☐ eigenvalues
- ☐ eigenvectors
- ☐ spectrum
- ☐ index
- ☐ perturbation class
- ☐ condition under which index stability applies

---

51. D11. K-Homology

- ☐ operator class

☐ Fredholm module metadata
☐ manifold/space assumptions
☐ singular-space framework
☐ external theorem reference
☐ partial/formal coverage

---

52. D12. Equivariant K-Theory

$$[\sigma(D)] \in K_G^0(T^*M)$$

☐ group action
☐ equivariant bundle
☐ equivariant operator
☐ equivariant symbol
☐ equivariant K-class
☐ equivariant index
☐ symmetry graph integration

---

53. D Acceptance Gate

설계도 D1-D6.

☐ D1 Ellipticity semantics
☐ D2 K-class display
☐ D3 Index semantics
☐ D4 Boundary semantics
☐ D5 Nonlinear bridge
☐ D6 Formal coverage

---

54. Adapter Architecture

설계도 공통 API:

```
Adapter.run(inputSpec, environment)
→
outputRepresentations
evidenceRecords
diagnostics
residuals/errorBounds
provenanceEdges
reproducibilityHash
```

모든 adapter 공통

- ☐ version
- ☐ input validation
- ☐ environment validation
- ☐ deterministic seed where possible
- ☐ outputs
- ☐ logs
- ☐ diagnostics
- ☐ residuals
- ☐ error bounds
- ☐ hashes
- ☐ cancellation
- ☐ timeout
- ☐ retry policy
- ☐ failure evidence

---

55. Worker / Job System

- ☐ job id
- ☐ queue
- ☐ startedAt
- ☐ completedAt
- ☐ input hash
- ☐ environment hash
- ☐ worker version
- ☐ status
- ☐ progress
- ☐ logs
- ☐ result
- ☐ cancellation
- ☐ stale-after-input-change handling

---

56. Computation Backend Integration

설계도 추천 후보.

ODE/DAE/nonlinear

- ☐ PETSc-like adapter or equivalent

PDE/FEM

- ☐ DOLFINx/FEniCS-like adapter or equivalent

Spectrum

- ☐ SLEPc
- ☐ ARPACK

☐ internal small matrix mode

Topology

☐ persistent homology adapter

3D

☐ Three.js/WebGL
☐ optional desktop renderer

Formal

☐ local Lean verifier
☐ cloud final verifier
☐ experimental LSP

---

57. Local-first Architecture

설계도 원칙.

☐ local compute default
☐ private research data stays local where possible
☐ cloud workers optional
☐ cloud job hashes saved
☐ local proof verification
☐ cloud proof verification optional
☐ Cloud LSP marked Experimental
☐ final verifier separate

---

58. `.mathscope` Bundle

설계도 구조.

```
project.mathscope/
 manifest.json
 objects/
 representations/
 operators/
 pde/
 spectra/
 geometry/
 topology/
 ktheory/
 claims/
 evidence/
 lean/
 numerics/
 views/
 screenshots/
 logs/
 environment.lock
```

Export

☐ all specs
☐ revision graph
☐ evidence
☐ solver configs
☐ meshes/data
☐ proof files
☐ hashes
☐ environment

Import

- ☐ schema version check
- ☐ migration
- ☐ missing file detection
- ☐ hash verification
- ☐ stale detection
- ☐ environment compatibility

---

59. Reproducibility

설계도는 screenshot만 저장해서는 안 된다고 명시한다.

- ☐ solver version
- ☐ adapter version
- ☐ OS/runtime
- ☐ Lean version
- ☐ mathlib commit
- ☐ input hashes
- ☐ mesh
- ☐ timestep
- ☐ tolerances
- ☐ truncation N
- ☐ basis
- ☐ random seed
- ☐ proof hash

---

60. Error & Uncertainty Panel

설계도 요구 메타데이터.

- ☐ residual norm
- ☐ mesh size
- ☐ timestep
- ☐ tolerance
- ☐ iteration count
- ☐ truncation dimension
- ☐ basis choice
- ☐ sampling density
- ☐ uncertainty
- ☐ theorem assumptions
- ☐ representation method
- ☐ display dimension

---

61. Regression Test Suite

설계도 페이지 23의 regression categories를 전부 executable test로 만든다.

Typing

- ☐ unbounded operator without domain → fail

Representation

- ☐ projection missing metadata → fail

PDE

- ☐ BC changed → old evidence stale

Spectrum

- ☐ finite spectrum labeled exact infinite → fail

Singularity

☐ numerical blow-up automatically theorem → fail

Topology

☐ similar 3D forms automatically equivalent → fail

AI

☐ assistant sets PROVED → fail

Lean

☐ LSP no-error => FORMAL PASS → fail

Index

☐ index 0 => invertible → fail

Boundary

☐ Atiyah-Bott obstruction => all BC impossible → fail

---

62. Formalization Coverage Matrix

설계도 Appendix A를 UI capability system으로 구현한다.
각 영역:

☐ `FULL`
☐ `PARTIAL`
☐ `EXTERNAL THEOREM`
☐ `NOT AVAILABLE`
☐ `UNKNOWN`

대상:

☐ finite linear algebra
☐ Banach/Hilbert
☐ self-adjoint spectrum
☐ Fredholm theory
☐ unbounded operator domains
☐ pseudodifferential calculus
☐ topological K-theory
☐ Atiyah-Singer
☐ Ricci flow
☐ Galois basics

---

63. Representative Demo Suite

설계도에서 네 개를 대표 end-to-end demo로 지정한다.

Demo 1 - S^3 Representation

- ☐ object S^3
- ☐ projection
- ☐ slice
- ☐ 3D rendering
- ☐ identifiability
- ☐ topology
- ☐ fidelity

목표:

> Representation ≠ Object

Demo 2 - Nonlinear PDE

- ☐ nonlinear PDE
- ☐ numerical candidate
- ☐ residual
- ☐ linearization
- ☐ spectrum
- ☐ singularity
- ☐ 3D projection
- ☐ stability claim

목표:

> Linearized ≠ Nonlinear theorem

Demo 3 - Symmetric Operator

- ☐ group action
- ☐ representation
- ☐ invariant subspace
- ☐ commuting operator
- ☐ eigenspaces
- ☐ spectral multiplicity
- ☐ Lean target

목표:

> Spectrum ↔ Symmetry requires explicit structure

Demo 4 - Elliptic Index

- ☐ PDE
- ☐ elliptic operator
- ☐ symbol
- ☐ ellipticity
- ☐ K-class
- ☐ analytic index
- ☐ topological index
- ☐ boundary obstruction
- ☐ Lean coverage

목표:

> Solve-before-solve structural analysis

---

64. 기준 Integration Test

설계도는 nonlinear elliptic PDE 세션을 기준 시나리오로 이미 정의한다.

$$F(u) = -\Delta_g u + \lambda u - u^3 = 0$$

이 하나를 **v0.3.1의 golden integration fixture**로 쓰자.

A

- ☐ PDESpec
- ☐ numerical u_*
- ☐ residual
- ☐ linearization
- ☐ spectrum

B

- ☐ field visualization
- ☐ candidate singularity
- ☐ nD→3D
- ☐ persistence
- ☐ fidelity

C

- ☐ claim
- ☐ assumptions
- ☐ stress test
- ☐ symmetry
- ☐ Lean target

D

- ☐ principal symbol
- ☐ ellipticity
- ☐ K-class
- ☐ index
- ☐ boundary
- ☐ spectral flow

이 fixture 하나가 처음부터 끝까지 완성되면 **MathScope의 핵심 architecture가 실제로 연결됐다고 볼 수 있다.**

---

65. UI Routes

설계도 navigation.

- ☐ /
- ☐ /workspace
- ☐ /docs
- ☐ /benchmarks
- ☐ /architecture
- ☐ /research

---

66. Homepage

첫 화면은 오히려 단순하게 유지.

```
MATHSCOPE

Mathematical objects are not their pictures.

Explore structures.
```

```
Measure what representation loses.
Certify what survives.

[Open Workspace]
[Try Demo]
```

설계도 homepage 원칙 그대로다.

---

67. Performance Checklist

이 부분은 설계도를 실제 구현으로 내리기 위한 **추가 엔지니어링 체크리스트**다.

- ☐ 3D canvas 60fps 목표
- ☐ large dataset progressive loading
- ☐ worker offloading
- ☐ long numerical job cancellation
- ☐ spectrum sampling throttle
- ☐ mesh LOD
- ☐ graph virtualization
- ☐ lazy tabs
- ☐ session autosave debounce
- ☐ incremental revision propagation

---

68. Failure UX

수학 소프트웨어는 성공보다 실패 표현이 중요하다.

- ☐ solver diverged
- ☐ mesh invalid
- ☐ theorem assumptions missing
- ☐ operator domain incomplete
- ☐ spectral pollution suspected
- ☐ formal dependency missing
- ☐ projection noninjective
- ☐ topology computation incomplete
- ☐ K-theory backend unavailable
- ☐ boundary analysis inconclusive

실패를 숨기지 않고 **Evidence로 저장**한다.

---

69. Documentation Checklist

- ☐ Getting Started
- ☐ Mathematical Contract
- ☐ ObjectSpec
- ☐ RepresentationSpec
- ☐ OperatorSpec
- ☐ PDESpec
- ☐ SpectrumSpec
- ☐ KTheorySpec
- ☐ EvidenceRecord
- ☐ status glossary
- ☐ solver guide
- ☐ proof guide
- ☐ projection/fidelity guide
- ☐ K-theory disclaimer
- ☐ AI boundary
- ☐ reproducibility

---

70. 최종 Release Gate

Product

☐ session create/save/load
☐ IDE stable
☐ four demos work
☐ `.mathscope` export/import
☐ docs available

Mathematical

☐ A1-A6 PASS
☐ B1-B6 PASS
☐ C1-C6 PASS
☐ D1-D6 PASS

Trust

☐ no automatic numerical→theorem promotion
☐ no finite→infinite conflation
☐ no visual→topological promotion
☐ no AI→proof promotion
☐ no index→invertibility promotion

Formal

☐ diagnostics separate
☐ final verifier separate
☐ pinned environment
☐ axiom audit
☐ coverage matrix

Reproducibility

☐ bundle replay
☐ environment replay
☐ numerical replay
☐ proof replay
☐ hash validation

권장 실제 개발 순서

여기서는 설계도 로드맵을 조금 더 개발 친화적으로 잘게 나누면 좋다.

Stage 1 - Foundation

`ResearchSession → schema → revision → evidence → IDE shell`

Stage 2 - Analytic vertical slice

`PDE → Operator → finite spectrum → 3D field`
이 단계에서 **처음부터 끝까지 움직이는 얇은 제품 하나**를 만든다.

Stage 3 - A 완성

Operator / PDE / Spectrum / Resolvent / nonlinear bridge

Stage 4 - B

Singularity / nD→3D / Fidelity / Topology

Stage 5 - C

Research Graph / Stress Test / Copilot / Proof Dock

Stage 6 - D

Symbol / Ellipticity / K-Class / Index / Boundary

Stage 7 - Advanced Research

Galois / Zeta / spectral flow / equivariant K-theory / Ricci flow

Stage 8 - Hardening

Regression / benchmark / bundle / docs / performance / accessibility

---

그리고 개발할 때 한 가지 원칙은 정말 중요해.

처음부터 200개 기능을 반쯤 구현하지 말고, 먼저 `Nonlinear Elliptic PDE` golden session 하나를 A→B→C→D까지 완전히 관통시키는 게 좋다. 설계도 자체도 이 시나리오를 integration fixture로 잡고 있고, candidate solution부터 linearization, spectrum, projection, stress test, symbol/K-class/index까지 연결한다.
그 하나가 제대로 작동하면 이후 Heat, Wave, Schrödinger, Ricci, Galois, zeta 같은 module은 **같은 contract 위에 adapter를 추가하는 작업**이 된다.
이 체크리스트 기준으로 보면 **v0.3.1은 약 70개 상위 작업군 + 수백 개의 세부 구현 항목** 규모야. 이제부터는 새 아이디어를 더 추가하기보다, 실제 구현에서 이 체크리스트를 **하나씩 PASS로 바꾸는 단계**로 들어가는 게 맞다.